Artifact for EMSOFT 2026: Suborbital and Orbital Real-Time Localization of Gamma-Ray Bursts via Utility-Guided Coordination

This repository contains the experiment drivers, C++ search-planning code, precomputed results, and visualization tools supporting the EMSOFT 2026 paper *Suborbital and Orbital Real-Time Localization of Gamma-Ray Bursts via Utility-Guided Coordination*.

- Repository: https://github.com/Daisy0419/GRB-Search-Pipeline
- Software artifact: Zenodo 10.5281/zenodo.21572120
- Artifact dataset: Zenodo 10.5281/zenodo.21497891
- Documentation: `REQUIREMENTS.md`, `INSTALL.md`, `STATUS.md`, and `LICENSE.md`
- Paper: `paper.pdf`

Table of Contents

Overview

This artifact runs offline experiments that simulate and evaluate the proposed on-board observation workflow. It does not acquire events from a live instrument or control a physical telescope.

The simulated runtime workflow gathers a mixed source-plus-background event stream, uses a trained utility model to choose when to generate a likelihood map, plans an optical search with Greedy Christofides Pathfinding (GCP), and simulates the search under a deadline using the telescope slew- and dwell-time models.

The artifact experiment has two phases:

1. **Training:** generate maps for all 52,800 short and 52,800 longlow training transients, measure mapping and GCP times, and build the empirical data used by the utility estimator.
2. **Evaluation:** select 10,000 test transients per scenario using seed `1957`, generate utility-guided and deadline-oblivious maps, run GCP, and measure whether the simulated search reaches the true source before each deadline.

The artifact provides three ways to evaluate the work:

Goal	Section	Approximate time
Visualize the distributed paper results	Section 2	Minutes
Regenerate training and evaluation results	Section 3	Approximately 39 hours
Rerun the platform-sensitive Figure 4 benchmark	Section 4	10--20 minutes per platform

Figures 1--3 are explanatory diagrams and are not generated by the artifact.

Workspace layout

The installation guide allows the workspace to reside on any sufficiently large filesystem. After completing `INSTALL.md`, the relevant layout is:

```
${ARTIFACT_ROOT}/
|-- GRB-Search-Pipeline/          # This artifact repository
|   |-- search-planning/          # C++ GCP implementation and tiling files
|   |-- results/                  # Outputs created by new experiment runs
|   |-- precomputed_results/      # Distributed paper results; do not
modify
|   |-- cosipy-312-deps.yaml      # ARM64 environment
|   |-- cosipy-312-intel.yml      # Figure-4-only Intel environment
|   `-- ...
|-- cosipy/                       # External dependency cloned by
INSTALL.md
|-- transients/                   # Models, simulated transients, benchmark
data
`-- dependencies/                # Miniconda, HDF5, HighFive, and LEMON
```

COSIpy is not part of the initial artifact checkout. It is an external dependency cloned and installed by `INSTALL.md`.

1 Installation and Path Configuration

1. Review `REQUIREMENTS.md`.
2. Complete `INSTALL.md`, including its data-download and smoke test sections.
3. In every new shell, configure the same workspace chosen during installation:

```
export ARTIFACT_ROOT="/absolute/path/to/emsoft-artifact"

export REPO_ROOT="${ARTIFACT_ROOT}/GRB-Search-Pipeline"
export COSIPY_ROOT="${ARTIFACT_ROOT}/cosipy"
export DATA_ROOT="${ARTIFACT_ROOT}/transients"
export DEPS_ROOT="${ARTIFACT_ROOT}/dependencies"
```

The full experiment workflow in Section 3 uses Linux ARM64 (`aarch64`) and the `cosipy-312` environment. Section 2 visualization can use either installed environment. The x86-64 environment

documented in INSTALL Section 7 is provided specifically for the Intel measurements in Figure 4.

2 Reproducing Paper Figures from Distributed Results

The CSV results used in the paper are distributed under `${REPO_ROOT}/precomputed_results/`. This directory should remain unchanged. Figures created by the reviewer are written under `${REPO_ROOT}/results/`.

Activate the environment installed for the current platform:

```
# ARM64 complete-workflow environment:
conda activate cosipy-312

# On x86-64, if only the optional Figure 4 environment was installed, use:
# conda activate cosipy-312-intel
```

Launch the notebook:

```
mkdir -p "${REPO_ROOT}/results/figures"
cd "${REPO_ROOT}/precomputed_results"

GRB_RESULTS_ROOT="${REPO_ROOT}/precomputed_results" \
GRB_FIGURE_OUTPUT_DIR="${REPO_ROOT}/results/figures" \
jupyter notebook visualize_Results.ipynb
```

The notebook reproduces Figures 4--11 from the distributed results. Figures 1--3 are explanatory diagrams.

3 Running the Full Experiments

The full workflow is computationally expensive and runs both Phase A and Phase B. It uses the ARM64 installation from INSTALL Sections 1--6.

3.1 Phase A: Generate Offline Training Data

Step 1: Configure the training run

Activate the ARM64 environment and set the numerical-library thread controls before starting Python:

```
conda activate cosipy-312

export MKL_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=8
export OMP_NUM_THREADS=1
export NUMEXPR_NUM_THREADS=1
```

`OPENBLAS_NUM_THREADS` controls NumPy matrix operations and is separate from the `-t 8` option passed to `map_adapt_transients.py`.

Configure input and output paths:

```
export MAP_SCRIPT="${COSIPY_ROOT}/cosipy/ts_map/map_adapt_transients.py"

export RESPONSE_MODEL="${DATA_ROOT}/models/adapt_response_w_area.h5"
export BACKGROUND_MODEL="${DATA_ROOT}/models/adapt_bkg_model.h5"

export SHORT_TRAINING_TRANSIENTS="${DATA_ROOT}/emsoft_training_short"
export LONGLOW_TRAINING_TRANSIENTS="${DATA_ROOT}/emsoft_training_longlow"
export SHORT_TEST_TRANSIENTS="${DATA_ROOT}/emsoft_test_short"
export LONGLOW_TEST_TRANSIENTS="${DATA_ROOT}/emsoft_test_longlow"

export TRAINING_ROOT="${REPO_ROOT}/results/training"
export TRAINING_MAP_ROOT="${TRAINING_ROOT}/maps"
export TRAINING_LOG_ROOT="${TRAINING_ROOT}/logs"

mkdir -p \
    "${TRAINING_MAP_ROOT}/short" \
    "${TRAINING_MAP_ROOT}/longlow" \
    "${TRAINING_LOG_ROOT}"
```

Step 2: Generate short-transient training maps (~4 hours)

```
env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    "${SHORT_TRAINING_TRANSIENTS}" \
    gt \
    -m \
    -o "${TRAINING_MAP_ROOT}/short" \
    > "${TRAINING_ROOT}/short_mapping_time_stats.csv" \
    2> "${TRAINING_LOG_ROOT}/short_mapping.log"
```

This command processes all 52,800 short training transients. Do not use `-s 10000` during training. The `gt` endpoint uses the true transient duration only to construct offline training data.

Step 3: Generate longlow-transient training maps (~4 hours)

```
env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
```

```
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    "${LONGLOW_TRAINING_TRANSIENTS}" \
    gt \
    -m \
    -o "${TRAINING_MAP_ROOT}/longlow" \
    > "${TRAINING_ROOT}/longlow_mapping_time_stats.csv" \
    2> "${TRAINING_LOG_ROOT}/longlow_mapping.log"
```

Training likelihood maps are independent of the optical FoV, so each transient is mapped only once.

Step 4: Run GCP on the training maps (~10 hours)

GCP is run for both scenarios (`short`, `longlow`) and both FoVs (`2.5x2.5`, `5.36x4.5`). The required tiling and true-source lookup files are distributed under `${REPO_ROOT}/search-planning/tilings/`.

The driver reads the exported `REPO_ROOT` variable. If it is not set, the repository root is inferred automatically from the location of the script. No source-code changes are required.

Keep the paper configuration:

```
FOVS = ["2.5x2.5", "5.36x4.5"]

TRAINING_MAP_DIRS = {
    "short": REPO_ROOT / "results" / "training" / "maps" / "short",
    "longlow": REPO_ROOT / "results" / "training" / "maps" / "longlow",
}

EXECUTABLE = SEARCH_ROOT / "build" / "sp_train"
RESULTS_DIR = REPO_ROOT / "results" / "training"
```

Make the user-built HDF5 library and Bitshuffle plugin available, then run the driver:

```
export HDF5_ROOT="${DEPS_ROOT}/hdf5"
export LD_LIBRARY_PATH="${HDF5_ROOT}/lib:${LD_LIBRARY_PATH:-}"
export HDF5_PLUGIN_PATH="$(
    python -c "import hdf5plugin; print(hdf5plugin.PLUGIN_PATH)"
)"

cd "${REPO_ROOT}/results"
python run_training.py
```

The four GCP training-output files are:

```
${REPO_ROOT}/results/training/
|-- short_searching_2.5x2.5_tiling.csv
```

```
|-- short_searching_5.36x4.5_tiling.csv
|-- longlow_searching_2.5x2.5_tiling.csv
`-- longlow_searching_5.36x4.5_tiling.csv
```

The mapping-time statistics and GCP outcomes serve different purposes. For example, `short_mapping_time_stats.csv` predicts mapping time, while `short_searching_5.36x4.5_tiling.csv` estimates the probability of reaching the source for a given search budget.

3.2 Phase B: Evaluate the Policies

Evaluation uses 10,000 transients from each test scenario. Seed `1957` selects the same subset for every policy and deadline.

If Phase B is started in a new shell, repeat Section 1's workspace exports and Phase A Step 1's environment and path configuration before continuing. Phase B uses the variables defined there, including `MAP_SCRIPT`, the model and test-data paths, and `TRAINING_ROOT`.

Step 5: Configure evaluation outputs

```
export VALIDATION_ROOT="${REPO_ROOT}/results/validation"
export VALIDATION_MAP_ROOT="${VALIDATION_ROOT}/maps"
export VALIDATION_STATS_ROOT="${VALIDATION_ROOT}/mapping_stats"
export VALIDATION_LOG_ROOT="${VALIDATION_ROOT}/logs"

mkdir -p \
    "${VALIDATION_MAP_ROOT}" \
    "${VALIDATION_STATS_ROOT}" \
    "${VALIDATION_LOG_ROOT}"

export SHORT_MAPPING_TIMES="${TRAINING_ROOT}/short_mapping_time_stats.csv"
export
LONGLOW_MAPPING_TIMES="${TRAINING_ROOT}/longlow_mapping_time_stats.csv"
export TRAINING_OUTCOMES_ROOT="${TRAINING_ROOT}"
```

The output case names match those under `${REPO_ROOT}/precomputed_results/validation/`, allowing the same notebook to read either result tree.

Step 6: Generate utility-guided test maps (~10 hours)

Utility-guided mapping depends on FoV because each FoV uses a different GCP training-outcome table. Generate one map set for every FoV and deadline.

Short-transient runs:

```
for FOV in 2.5x2.5 5.36x4.5; do
    for DEADLINE in 10 15 20 25 30 35 40 45 50; do
        CASE_NAME="${FOV}_short"
```

```
        MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
        STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
        LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

        mkdir -p \
            "${MAP_CASE_ROOT}" \
            "${STATS_CASE_ROOT}" \
            "${LOG_CASE_ROOT}"

        echo "Running short: FoV=${FOV}, deadline=${DEADLINE}"

        env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
        python "${MAP_SCRIPT}" \
            --response "${RESPONSE_MODEL}" \
            --bkg-model "${BACKGROUND_MODEL}" \
            -t 8 \
            -n 64 \
            -s 10000 \
            -r 1957 \
            --training-times "${SHORT_MAPPING_TIMES}" \
            --training-outcomes \

"${TRAINING_OUTCOMES_ROOT}/short_searching_${FOV}_tiling.csv" \
            "${SHORT_TEST_TRANSIENTS}" \
            "${DEADLINE}" \
            -m \
            -o "${MAP_CASE_ROOT}/emsoft_maps_${DEADLINE}" \
            > "${STATS_CASE_ROOT}/emsoft_stats_${DEADLINE}.csv" \
            2> "${LOG_CASE_ROOT}/emsoft_maps_${DEADLINE}.log"
    done
done
```

Longlow-transient runs:

```
for FOV in 2.5x2.5 5.36x4.5; do
    for DEADLINE in 30 40 50 60 70 80 90 100 110; do
        CASE_NAME="${FOV}_longlow"
        MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
        STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
        LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

        mkdir -p \
            "${MAP_CASE_ROOT}" \
            "${STATS_CASE_ROOT}" \
            "${LOG_CASE_ROOT}"

        echo "Running longlow: FoV=${FOV}, deadline=${DEADLINE}"

        env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
        python "${MAP_SCRIPT}" \
            --response "${RESPONSE_MODEL}" \
            --bkg-model "${BACKGROUND_MODEL}" \
```

```
            -t 8 \
            -n 64 \
            -s 10000 \
            -r 1957 \
            --training-times "${LONGLOW_MAPPING_TIMES}" \
            --training-outcomes \

"${TRAINING_OUTCOMES_ROOT}/longlow_searching_${FOV}_tiling.csv" \
            "${LONGLOW_TEST_TRANSIENTS}" \
            "${DEADLINE}" \
            -m \
            -o "${MAP_CASE_ROOT}/emsoft_maps_${DEADLINE}" \
            > "${STATS_CASE_ROOT}/emsoft_stats_${DEADLINE}.csv" \
            2> "${LOG_CASE_ROOT}/emsoft_maps_${DEADLINE}.log"
    done
done
```

Step 7: Generate deadline-oblivious test maps (~1 hour)

The `nodeadline` policy does not use the utility tables and does not depend on FoV. Generate it once for each scenario and reuse it for both FoVs and every deadline.

Short-transient baseline:

```
CASE_NAME="no-fov_short"
MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

mkdir -p "${MAP_CASE_ROOT}" "${STATS_CASE_ROOT}" "${LOG_CASE_ROOT}"

env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    -s 10000 \
    -r 1957 \
    "${SHORT_TEST_TRANSIENTS}" \
    nodeadline \
    -m \
    -o "${MAP_CASE_ROOT}/emsoft_maps_nodeadline" \
    > "${STATS_CASE_ROOT}/emsoft_stats_nodeadline.csv" \
    2> "${LOG_CASE_ROOT}/emsoft_maps_nodeadline.log"
```

Longlow-transient baseline:

```
CASE_NAME="no-fov_longlow"
MAP_CASE_ROOT="${VALIDATION_MAP_ROOT}/${CASE_NAME}"
STATS_CASE_ROOT="${VALIDATION_STATS_ROOT}/${CASE_NAME}"
LOG_CASE_ROOT="${VALIDATION_LOG_ROOT}/${CASE_NAME}"

mkdir -p "${MAP_CASE_ROOT}" "${STATS_CASE_ROOT}" "${LOG_CASE_ROOT}"

env -u LD_LIBRARY_PATH -u HDF5_PLUGIN_PATH \
python "${MAP_SCRIPT}" \
    --response "${RESPONSE_MODEL}" \
    --bkg-model "${BACKGROUND_MODEL}" \
    -t 8 \
    -n 64 \
    -s 10000 \
    -r 1957 \
    "${LONGLOW_TEST_TRANSIENTS}" \
    nodeadline \
    -m \
    -o "${MAP_CASE_ROOT}/emsoft_maps_nodeadline" \
    > "${STATS_CASE_ROOT}/emsoft_stats_nodeadline.csv" \
    2> "${LOG_CASE_ROOT}/emsoft_maps_nodeadline.log"
```

The maps, mapping statistics, and logs use these six case names:

```
2.5x2.5_longlow
2.5x2.5_short
5.36x4.5_longlow
5.36x4.5_short
no-fov_longlow
no-fov_short
```

Step 8: Run GCP and simulate the search (~10 hours)

The driver reads the exported `REPO_ROOT` variable. If it is not set, the repository root is inferred automatically from the location of the script. No source-code changes are required.

Keep the paper configuration:

```
SCENARIOS = ["short", "longlow"]
POLICIES = ["utility", "nodeadline"]
FOVS = ["2.5x2.5", "5.36x4.5"]

DEADLINES = {
    "short": [10, 15, 20, 25, 30, 35, 40, 45, 50],
    "longlow": [30, 40, 50, 60, 70, 80, 90, 100, 110],
}
```

Make HDF5 and the Bitshuffle plugin available and run the driver:

```
export HDF5_ROOT="${DEPS_ROOT}/hdf5"
export LD_LIBRARY_PATH="${HDF5_ROOT}/lib:${LD_LIBRARY_PATH:-}"
export HDF5_PLUGIN_PATH="$(
    python -c "import hdf5plugin; print(hdf5plugin.PLUGIN_PATH)"
)"

cd "${REPO_ROOT}/results"
python run_validation.py
```

For each generated map, the driver subtracts gathering, mapping, and GCP planning time from the overall deadline, simulates the remaining optical search, and records whether the source tile is reached.

Search results are grouped by FoV:

```
${REPO_ROOT}/results/validation/
|-- searching_results_2.5x2.5/
`-- searching_results_5.36x4.5/
```

Each CSV is named:

```
<scenario>_<fov>_tiling_<policy>_<deadline>.csv
```

Step 9: Aggregate and visualize recomputed results (~10 minutes)

```
conda activate cosipy-312
mkdir -p "${REPO_ROOT}/results/figures"
cd "${REPO_ROOT}/precomputed_results"

GRB_RESULTS_ROOT="${REPO_ROOT}/results" \
GRB_FIGURE_OUTPUT_DIR="${REPO_ROOT}/results/figures" \
jupyter notebook visualize_Results.ipynb
```

The notebook reads recomputed files under `${REPO_ROOT}/results/` and writes new figures under `${REPO_ROOT}/results/figures/`.

4 Running the Mapping-Performance Benchmark (Optional)

This optional benchmark reproduces the mapping-time measurements in Figure 4. Figure 4 contains five measurements on a 2.3 GHz Intel Xeon Gold 5118 system and three measurements on the Jetson Orin NX.

Benchmark mode	Figure 4 label	Intel	Jetson

Benchmark mode	Figure 4 label	Intel	Jetson
`cosipy_interp`	`cosipy v3`	yes	--
`cosipy_numba`	`cosipy + JIT`	yes	--
`emsoft_outmem`	`ext-mem`	yes	yes
`emsoft_inmem`	`in-mem`	yes	yes
`emsoft_moc`	`in-mem-mr`	yes	yes

The two original COSIpy modes require more memory than is available on the 16 GB Jetson and therefore appear only in the Intel results.

Environment installation is intentionally kept out of this optional experiment section:

- For Intel x86-64, complete INSTALL Section 7. That environment is used only for the Intel measurements in Figure 4.
- For Jetson ARM64, use the main ARM64 environment from INSTALL Section 3.

Each benchmark uses eight CPU cores and reports the average of five measured iterations after one warm-up iteration:

```
TIME: 0.402 s
```

4.1 Intel x86-64 benchmarks (~20 minutes)

Activate the Figure-4-only Intel environment and configure the benchmark:

```
conda activate cosipy-312-intel
cd "${COSIPY_ROOT}/cosipy/ts_map"

export DC3_BENCHMARK_DATA="${DATA_ROOT}/dc3_benchmark_data"
export BENCHMARK_RESULTS="${REPO_ROOT}/results/dc3_benchmark/intel"
mkdir -p "${BENCHMARK_RESULTS}"

export MKL_NUM_THREADS=1
export OMP_NUM_THREADS=1
export NUMEXPR_NUM_THREADS=1
```

Run the two original COSIpy modes with one OpenBLAS thread. These modes use eight Python processes, with one OpenBLAS thread in each process:

```
export OPENBLAS_NUM_THREADS=1

for MODE in cosipy_interp cosipy_numba; do
    echo "Running ${MODE}"
```

```
    python dc3_benchmark.py \
        -d "${DC3_BENCHMARK_DATA}" \
        "${MODE}" \
        | tee "${BENCHMARK_RESULTS}/${MODE}.txt"
done
```

Run the three EMSOFT modes with eight OpenBLAS threads:

```
export OPENBLAS_NUM_THREADS=8

for MODE in emsoft_outmem emsoft_inmem emsoft_moc; do
    echo "Running ${MODE}"
    python dc3_benchmark.py \
        -d "${DC3_BENCHMARK_DATA}" \
        "${MODE}" \
        | tee "${BENCHMARK_RESULTS}/${MODE}.txt"
done
```

4.2 Jetson ARM64 benchmarks (~10 minutes)

```
conda activate cosipy-312
cd "${COSIPY_ROOT}/cosipy/ts_map"

export DC3_BENCHMARK_DATA="${DATA_ROOT}/dc3_benchmark_data"
export BENCHMARK_RESULTS="${REPO_ROOT}/results/dc3_benchmark/jetson"
mkdir -p "${BENCHMARK_RESULTS}"

export MKL_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=8
export OMP_NUM_THREADS=1
export NUMEXPR_NUM_THREADS=1

for MODE in emsoft_outmem emsoft_inmem emsoft_moc; do
    echo "Running ${MODE}"
    python dc3_benchmark.py \
        -d "${DC3_BENCHMARK_DATA}" \
        "${MODE}" \
        | tee "${BENCHMARK_RESULTS}/${MODE}.txt"
done
```

4.3 Visualize the benchmark results

Display the measured times:

```
grep "TIME:" "${REPO_ROOT}"/results/dc3_benchmark/intel/*.txt
grep "TIME:" "${REPO_ROOT}"/results/dc3_benchmark/jetson/*.txt
```

It is valid to run only the platform available to the reviewer; omit the corresponding `grep` command if that result directory does not exist.

Launch the distributed-results notebook with the environment installed for the current platform:

```
cd "${REPO_ROOT}/precomputed_results"

GRB_RESULTS_ROOT="${REPO_ROOT}/precomputed_results" \
GRB_FIGURE_OUTPUT_DIR="${REPO_ROOT}/results/figures" \
jupyter notebook visualize_Results.ipynb
```

In the notebook, find **Optional: Reproduce Figure 4** and copy each reported `TIME:` value into `time_means`. Each tuple has the form `(Intel_time, Jetson_time)` in seconds. Keep `np.nan` for the two original COSIpy modes that are not run on the Jetson.